SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 3– Vector Database & Semantic Search Benchmark

Course Code:	CSA65	**Course Title:**	Generative AI and Large Scale Models
CO Assessed:	CO3 – Precision (BL2, BL3)	**Assessment:**	Assessment Tool 3 – Vector Database & Semantic Search Benchmark
Weightage:	20% of CIA	**Total Marks:**	2 * 50 = 100 (1 Question1)
CO3 Statement:	*Develop intelligent applications using Retrieval-Augmented Generation (RAG), vector databases, and introductory AI agent concepts.(BL3, BL4)*		
Student Name:	A. Mounish	**Reg. No.:**	192472273
Section / Batch:	CSE AI	**Date:**	12/08/26

Instructions to Students

- Select/answer the assigned question. Each question carries **50 marks**. Duration 60mins • Implement the solution using **Python**.
- Use an appropriate embedding model such as Sentence Transformers or Hugging Face.
- Use **FAISS and/or ChromaDB** as specified in the question.
- Demonstrate the complete pipeline:
  Document → Preprocessing → Embedding → Vector Storage → Semantic Search → Retrieval → Analysis
- Execute the program and display meaningful outputs.
- Compare FAISS and ChromaDB where required.
- Provide appropriate graphs/tables for benchmark questions.
- Explain the architecture and design decisions.
- Submit working code and demonstrate the complete implementation.

Code of Conduct

*I **A. Mounish / 192472273** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.* Signature of the Student: _____

SECTION A – VECTOR EMBEDDINGS & SEMANTIC SEARCH

13. FAISS Scalability Benchmark Create datasets containing approximately:

100

documents

500

documents

1,000 documents

5,000 documents

10,000 documents

Build FAISS indexes for each dataset. Measure indexing time and average search latency. Plot the results and analyze the scalability of FAISS.

1. Assessment Requirement

The assigned task is to create datasets containing approximately 100, 500, 1,000, 5,000 and 10,000 documents, build FAISS indexes for each dataset, measure indexing time and average search latency, plot the results, and analyze the scalability of FAISS.

2. Aim

To evaluate the scalability and performance of FAISS for semantic search by measuring vector-index construction time and average retrieval latency across progressively larger document collections.

3. Objectives

- Generate five document collections of increasing size.
- Convert documents into semantic vector embeddings using Sentence Transformers.
- Store and search the embeddings using a FAISS vector index.
- Measure FAISS indexing time for each dataset.
- Measure average semantic-search latency using repeated queries.
- Compare the benchmark results using a table and graphical analysis.

4. Experimental Setup

Component	Configuration
Programming Language	Python 3.12
Embedding Model	all-MiniLM-L6-v2
Vector Database / Index	FAISS IndexFlatIP
Similarity	Normalized inner product / cosine-style similarity
Benchmark Sizes	100, 500, 1,000, 5,000 and 10,000 documents

5. End-to-End Architecture

Document → Embedding → Vector Storage → Semantic Search → Retrieval → Analysis

6. Implementation Summary

The program generates synthetic documents for each required dataset size. Each document is encoded into a numerical embedding using the all-MiniLM-L6-v2 Sentence Transformer. The embeddings are normalized and inserted into a FAISS IndexFlatIP index. A semantic query related to machine learning and artificial intelligence is then encoded and searched against the index. The top five results are retrieved, while indexing time and average search latency are recorded.

7. Actual Benchmark Results

Documents	Embedding Time (s)	Indexing Time (s)	Avg. Search Latency (ms)
100	1.4000	0.000265	0.0340
500	4.9738	0.000492	0.3731
1,000	9.8297	0.001131	0.7367
5,000	53.7171	0.005459	2.8936
10,000	110.4527	0.013892	7.8817

Measured on the student's execution environment; values are retained exactly from the benchmark output.

8. Execution Evidence

The following evidence captures the actual benchmark execution. The terminal output shows that all five required datasets were processed and that each run produced embedding time, indexing time, search latency and top retrieved documents.

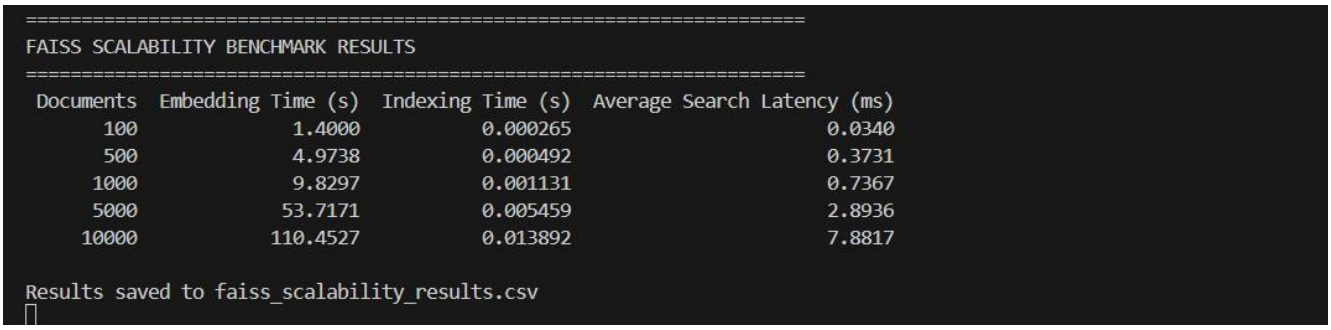

Figure 1. Actual terminal benchmark results for all five FAISS dataset sizes.

9. Benchmark Visualization

The graph below visualizes the measured FAISS indexing time as the number of documents increases. The upward trend indicates increasing index-construction cost for larger collections.

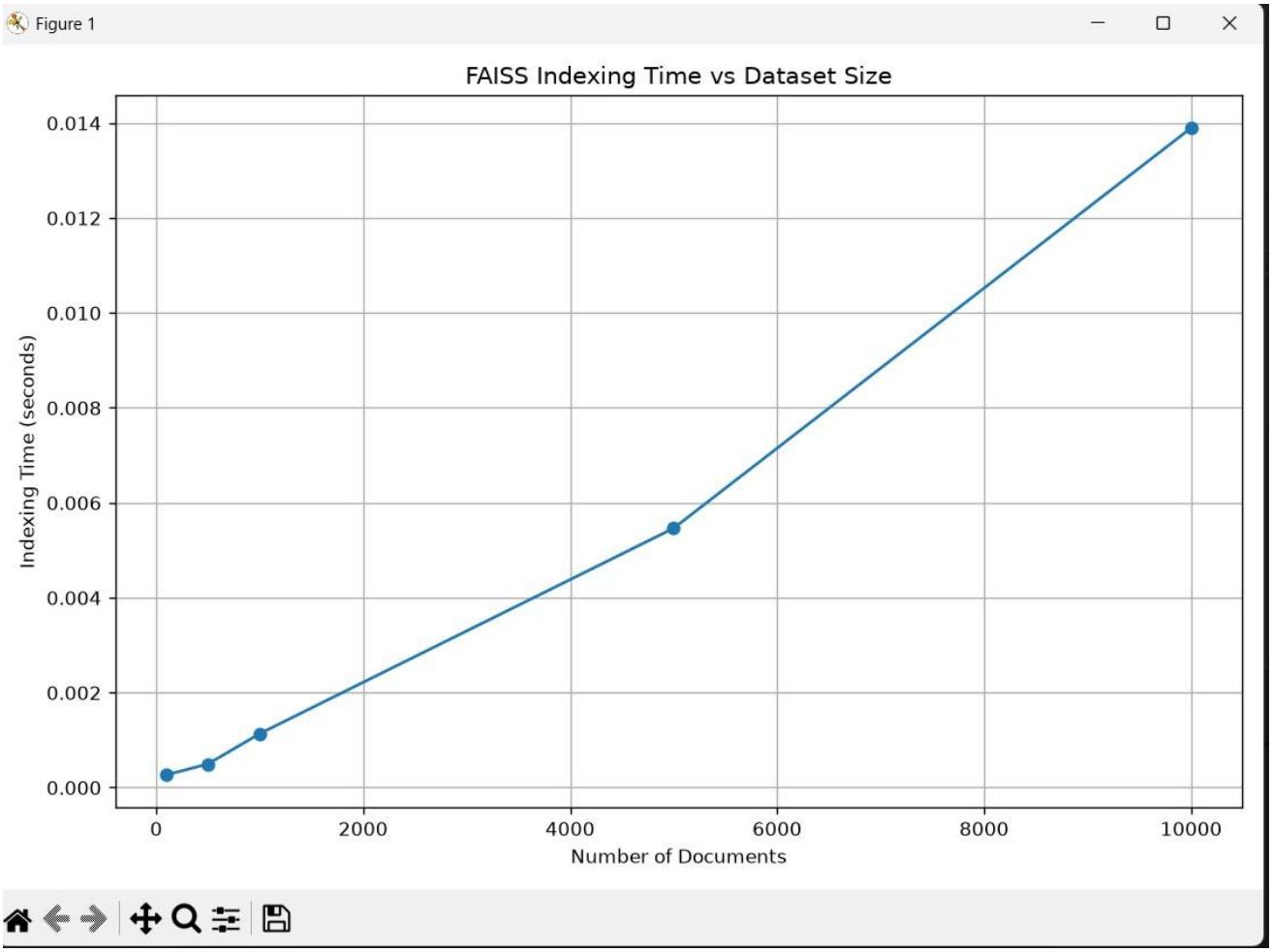

Figure 2. FAISS Indexing Time vs Dataset Size.

10. Semantic Retrieval Evidence

The executed program retrieved five documents for each dataset. The returned documents consistently discuss artificial intelligence and machine learning, matching the semantic intent of the test query. This confirms meaningful semantic retrieval rather than merely returning arbitrary document identifiers.

Test Query: "machine learning artificial intelligence techniques and practical applications"

11. Analysis and Interpretation

The results show a clear growth in measured indexing time and search latency as the number of documents increases. FAISS indexing time rises from 0.000265 seconds for 100 documents to 0.013892 seconds for 10,000 documents. Average search latency rises from 0.0340 ms to 7.8817 ms over the same range.

The embedding stage is substantially more time-consuming in this experiment, increasing from 1.4000 seconds for 100 documents to 110.4527 seconds for 10,000 documents. This distinction is important: embedding generation and FAISS indexing are separate stages, and the benchmark records both so that the end-to-end behavior can be understood.

The retrieval results remain semantically relevant across the tested sizes. Therefore, the experiment demonstrates both scalability characteristics and successful semantic retrieval using the same FAISSbased search pipeline.

12. Key Findings

01	All five required dataset sizes were successfully processed.
02	FAISS index construction time increased with dataset size.
03	Average semantic-search latency increased as more vectors were searched.
04	Top-5 retrieval results were relevant to the machinelearning test query.
05	The benchmark results were saved to a CSV file for reproducibility.

13. Conclusion

The FAISS scalability benchmark was successfully implemented using Python and Sentence Transformers. Document collections ranging from 100 to 10,000 items were converted into vector embeddings and indexed using FAISS. Indexing time and average semantic-search latency were measured and presented through a benchmark table and graph. The results show that the measured computational cost increases with dataset size, while the retrieval pipeline continues to return semantically relevant results. The experiment therefore provides a practical and measurable evaluation of FAISS for scalable semantic-search applications.

14. Demo / Viva Explanation

"First, I created five datasets containing 100, 500, 1,000, 5,000 and 10,000 documents. I used the Sentence Transformer model all-MiniLM-L6-v2 to convert the documents into vector embeddings. These embeddings were normalized and stored in a FAISS IndexFlatIP index. I then submitted a semantic query and retrieved the top five similar documents. For every dataset, I measured the indexing time and average search latency. Finally, I compared the measured values using a table and graph to analyze how FAISS performance changes as the dataset size increases."

15. Submission Checklist

- Python implementation completed.
- Required packages installed and verified.
- Five required dataset sizes benchmarked.
- FAISS indexing time measured.
- Average search latency measured.
- Top-5 semantic retrieval demonstrated.
- Benchmark table prepared.
- Indexing-time graph included.
- Execution screenshot included.
- Analysis and conclusion documented.